



Git Inside MetaEditor



MetaTrader 5 / Торговые системы

За пределами GARCH (Часть VI): Фракционное броуновское движение и мультипликативный каскад в MQL5

0

Muhammad Minhas Qamar

71

Введение

В [четвертой части](#) мы создали модуль анализа разбиений, который определяет функцию масштабирования $\tau(q)$ и показатель Херста по исходным ценовым данным. В [пятой части](#) мы добавили модуль подгонки спектра, который по $\tau(q)$ подбирает вероятностное распределение и его параметры. С помощью этих двух модулей мы ответили на вопрос: "Каковы параметры модели?" В этой статье мы ответим на вопрос: "Что с ними делать?"

Теперь мы создаем модуль моделирования: он использует параметры, полученные при подгонке (H , тип распределения, коэффициенты распределения, выборочную волатильность), и генерирует полную синтетическую ценовую траекторию MMAR. Работа модуля состоит из трех этапов, соответствующих компонентам мультифрактальной модели доходности активов:

1. мультипликативный каскад — генерирует мультифрактальное торговое время $\theta(t)$, "деформированные часы", преобразующие равномерное время в рыночное с кластеризацией волатильности;
2. фракционное броуновское движение — генерирует гауссовский процесс $B_H(t)$ с долгой памятью методом Дэвиса — Харта (БПФ) или разложением Холецкого;
3. деформация времени — объединяет эти компоненты в виде $X(t) = B_H[\theta(t)]$, что дает синтетические доходности с долгой памятью и мультифрактальным масштабированием.

Мы рассмотрим следующие темы:

1. [Архитектура CSimulationEngine](#)
2. [Построение мультипликативного каскада](#)
3. [Генерация фракционного броуновского движения](#)
4. [Композиция процесса MMAR](#)
5. [Проверка моделирования](#)
6. [Заключение](#)

Архитектура CSimulationEngine

Модуль моделирования реализован в одном классе CSimulationEngine, объединяющем весь пайплайн генерации. Класс намеренно спроектирован так, чтобы не хранить состояние между запусками: вы инициализируете его параметрами, вызываете метод Generate() и извлекаете результаты. Благодаря этому можно безопасно создавать множество экземпляров параллельно — именно так будет работать модуль Монте-Карло в седьмой части.

Прежде чем рассматривать сам класс, посмотрим, какие компоненты мы подключаем. Модуль моделирования широко использует стандартную библиотеку MetaTrader 5. Это важно подчеркнуть: не

все читатели знают, какие средства уже входят в поставку платформы. Для метода Дэвиса — Харта мы используем [быстрое преобразование Фурье из ALGLIB](#) (CFastFourierTransform), для разложения Холецкого — [средства линейной алгебры ALGLIB](#) (CTrFac, CMatrixDouble), а для генерации случайных чисел из нормального, гамма- и пуассоновского распределений — библиотеку [Math\Stat](#). Все эти компоненты входят в поставку MetaTrader 5 — внешние зависимости не требуются:

```
#include "MMARConstants.mqh"
#include <Math\Alglib\fasttransforms.mqh>
#include <Math\Alglib\linalg.mqh>
#include <Math\Stat\Normal.mqh>
#include <Math\Stat\Gamma.mqh>
#include <Math\Stat\Poisson.mqh>
```

Первая директива include подключает наш файл констант из четвертой части. Остальные пять подключений относятся к стандартной библиотеке MetaTrader 5. Файл fasttransforms.mqh содержит функции FFTR1D (прямое БПФ из вещественных данных в комплексные) и FFTC1DInv (обратное комплексное БПФ), используемые в алгоритме Дэвиса — Харта. Файл linalg.mqh предоставляет CTrFac::SPDMatrixCholesky для разложения симметричных положительно определенных матриц и CMatrixDouble как контейнер для матриц. Три заголовочных файла Math\Stat содержат функции [MathRandomNormal](#), [MathRandomGamma](#) и [MathRandomPoisson](#) соответственно. Это надежные реализации промышленного уровня: они обрабатывают пограничные случаи и генерируют выборки с хорошими статистическими свойствами. Любому разработчику на языке MQL5 полезно знать возможности стандартной библиотеки — многие из ее инструментов используются реже, чем заслуживают.

Определив зависимости, рассмотрим объявление класса:

```
//+-----+
//| CSimulationEngine class |
//+-----+
class CSimulationEngine
{
private:
    ///--- Model parameters
    double m_H;
    ENUM_MMAR_DISTRIBUTION m_dist_type;
    double m_dist_params[4];
    double m_sample_volatility;
    int m_cascade_b;
    int m_cascade_k;
    int m_n_points;

    ///--- Generated data
    double m_measure[];
    double m_trading_time[];
    double m_fbm[];
    double m_fbm_increments[];
    double m_mmar_process[];
    double m_mmar_returns[];

    bool m_initialized;

    ///--- Cascade generation
    void SampleMultipliers(double &multipliers[], int n_samples);
    bool GenerateCascade(void);
    void IntegrateMeasure(void);
```

```

//--- FBM generation
double          FBMAutocovariance(int k);
bool            GenerateFBM_DaviesHarte(void);
bool            GenerateFBM_Cholesky(void);
void            ScaleFBM(void);

//--- Composition
void            ComposeMMARProcess(void);

public:

                CSimulationEngine(void);
                ~CSimulationEngine(void);

//--- Initialization
bool            Init(double H, ENUM_MMAR_DISTRIBUTION dist_type,
                    const double &dist_params[], double sample_volatility,
                    int cascade_b, int cascade_k);

//--- Generation
bool            Generate(void);

//--- Output access
void            GetReturns(double &returns[], int &n_returns);
void            GetProcess(double &process[]);
void            GetTradingTime(double &theta[]);
void            GetFBM(double &fbm[]);
void            GetMeasure(double &measure[]);
double          GetVolatility(void);
int             GetNumPoints(void) { return m_n_points; }
};

```

Класс хранит в массивах-полях все промежуточные результаты — исходную каскадную меру, накопленное торговое время, траекторию FBM и итоговый составной процесс. Это сделано намеренно: внешний код может отдельно исследовать результаты каждого этапа для проверки и отладки. Число сгенерированных точек всегда равно b^k — основанию каскада, возведенному в степень его глубины. При значениях по умолчанию $b = 2$ и $k = 10$ получаем 1 024 точки на каждую моделируемую траекторию.

Метод Init() проверяет все параметры перед выделением памяти:

```

//+-----+
//| Init - Validate parameters and allocate arrays |
//+-----+
bool CSimulationEngine::Init(double H, ENUM_MMAR_DISTRIBUTION dist_type,
                             const double &dist_params[], double sample_volatility,
                             int cascade_b, int cascade_k)
{
    if(H <= 0.0 || H >= 1.0)
    {
        Print("SimulationEngine::Init - H must be in (0, 1), got ", H);
        return false;
    }
    if(cascade_b < 2)
    {
        Print("SimulationEngine::Init - cascade_b must be >= 2");
        return false;
    }
}

```

```

if(cascade_k < 1)
{
    Print("SimulationEngine::Init - cascade_k must be >= 1");
    return false;
}
if(sample_volatility <= 0.0)
{
    Print("SimulationEngine::Init - sample_volatility must be > 0");
    return false;
}

m_H = H;
m_dist_type = dist_type;
m_sample_volatility = sample_volatility;
m_cascade_b = cascade_b;
m_cascade_k = cascade_k;

for(int i = 0; i < 4; i++)
    m_dist_params[i] = (i < ArraySize(dist_params)) ? dist_params[i] : 0.0;

m_n_points = (int)MathPow(cascade_b, cascade_k);

ArrayResize(m_measure, m_n_points);
ArrayResize(m_trading_time, m_n_points);
ArrayResize(m_fbm, m_n_points);
ArrayResize(m_fbm_increments, m_n_points);
ArrayResize(m_mmar_process, m_n_points);
ArrayResize(m_mmar_returns, m_n_points - 1);

m_initialized = true;
return true;
}

```

Параметры поступают непосредственно из двух предыдущих модулей: H — из анализа разбиений, $dist_type$ и $dist_params$ — из модуля подгонки спектра, $sample_volatility$ — из исходного ряда доходностей. В нашей реализации коэффициент ветвления $cascade_b$ всегда равен 2 — каскад двоичный. Параметр глубины $cascade_k$ определяет разрешение — общее число точек b^k . Затем метод `Generate()` последовательно выполняет три этапа:

```

//+-----+
//| Generate - Run the full simulation pipeline |
//+-----+
bool CSimulationEngine::Generate(void)
{
    if(!m_initialized)
    {
        Print("SimulationEngine::Generate - not initialized");
        return false;
    }

    if(!GenerateCascade())
    {
        Print("SimulationEngine::Generate - cascade generation failed");
        return false;
    }

    IntegrateMeasure();
}

```

```

bool fbm_ok;
if(m_n_points <= 1024)
    fbm_ok = GenerateFBM_Cholesky();
else
    fbm_ok = GenerateFBM_DaviesHarte();

if(!fbm_ok)
{
    if(m_n_points <= 1024)
        fbm_ok = GenerateFBM_DaviesHarte();
    else
        Print("SimulationEngine::Generate - FBM generation failed");

    if(!fbm_ok)
        return false;
}

ScaleFBM();
ComposeMMARProcess();

return true;
}

```

Метод генерации FBM выбирается адаптивно: для коротких траекторий из 1 024 точек или меньше мы предпочитаем метод Холецкого — он точен, а затраты $O(n^3)$ еще приемлемы. Для более длинных траекторий используем метод Дэвиса — Харта со сложностью $O(n \log n)$ благодаря БПФ. Если основной метод не срабатывает, переходим к альтернативному. Для метода Холецкого причиной сбоя может быть потеря положительной определенности ковариационной матрицы из-за численной погрешности, для метода Дэвиса — Харта — отрицательные собственные значения.

Построение мультипликативного каскада

Мультипликативный каскад создает мультифрактальное торговое время. Единичный интервал последовательно делится на все более мелкие части; каждой присваивается масса с помощью случайных множителей из распределения, полученного при подгонке. Получается крайне неравномерная мера: в одних интервалах сосредоточена большая масса (периоды высокой активности), а в других ее почти нет (периоды затишья). Накопленная сумма меры дает $\theta(t)$ — торговое время, которое изменит временную шкалу FBM.

Выборка множителей

Распределение, выбранное в пятой части, определяет способ генерации весов каскада. Каждый множитель M связан со случайной величиной V формулой $M = b^{(-V)}$; V выбирается из распределения, полученного при подгонке. Это отображение преобразует положительную случайную величину в множитель от почти нулевого (большое $V \rightarrow$ очень малая масса) до очень большого (V около нуля или отрицательное $\rightarrow$ сконцентрированная масса):

```

//+-----+
//| SampleMultipliers - Draw cascade weights from chosen distribution|
//+-----+
void CSimulationEngine::SampleMultipliers(double &multipliers[], int n_samples)
{
    double V[];
    ArrayResize(V, n_samples);

```



```

switch(m_dist_type)
{
    case MMAR_DIST_NORMAL:
    {
        double alpha_0 = m_dist_params[0];
        double mu = alpha_0 / m_H;
        double variance = 2.0 * (alpha_0 / m_H - 1.0) * MathLog(m_cascade_b);
        double sigma = MathSqrt(MathMax(variance, 0.01));
        MathRandomNormal(mu, sigma, n_samples, V);
        break;
    }
    case MMAR_DIST_BINOMIAL:
    {
        double alpha_min = m_dist_params[0];
        double alpha_max = m_dist_params[1];
        for(int i = 0; i < n_samples; i++)
            V[i] = (MathRand() / 32767.0 < 0.5) ? alpha_min : alpha_max;
        break;
    }
    case MMAR_DIST_POISSON:
    {
        double alpha_0 = m_dist_params[0];
        double lambda = alpha_0 / m_H;
        MathRandomPoisson(lambda, n_samples, V);
        break;
    }
    case MMAR_DIST_GAMMA:
    {
        double alpha_0 = m_dist_params[0];
        double gamma_param = m_dist_params[1];
        double beta = MathLog(m_cascade_b) / (MathPow(m_cascade_b, 1.0 / gamma_param));
        double scale = 1.0 / beta;
        MathRandomGamma(gamma_param, scale, n_samples, V);
        break;
    }
}

ArrayResize(multipliers, n_samples);
for(int i = 0; i < n_samples; i++)
{
    multipliers[i] = MathPow((double)m_cascade_b, -V[i]);
    multipliers[i] = MathMax(1e-10, MathMin(multipliers[i], 1e10));
}

//--- Normalize within groups of b to sum to b (conservation of mass)
for(int i = 0; i < n_samples; i += m_cascade_b)
{
    double sum = 0.0;
    int group_end = MathMin(i + m_cascade_b, n_samples);
    for(int j = i; j < group_end; j++)
        sum += multipliers[j];
    if(sum > 0.0)
    {
        for(int j = i; j < group_end; j++)
            multipliers[j] *= (double)m_cascade_b / sum;
    }
}

```

```
}  
}
```

Для каждого распределения величина V получается из его параметров своим способом. В нормальном распределении используются $\mu = \alpha_0/N$ и дисперсия, полученная из теоретической связи между α_0 , N и основанием каскада. Биномиальный вариант самый простой: $\alpha_{\min}$ или $\alpha_{\max}$ выбирается случайно с равной вероятностью. В пуассоновском варианте генерируются дискретные значения с математическим ожиданием α_0/N . Для гамма-распределения используются параметр формы γ и масштаб, определяемый основанием каскада. Во всех случаях преобразование $M = b^{(-V)}$ дает положительный множитель, а ограничение диапазоном $[1e-10, 1e10]$ предотвращает численное переполнение.

Заключительная нормализация имеет решающее значение. Преобразовав V в множители, мы группируем их попарно (поскольку $b = 2$) и нормализуем каждую пару до суммы $b = 2$. Так обеспечивается сохранение массы: на каждом уровне каскада общая масса дочерних интервалов равна общей массе родительских, умноженной на b . Без этого ограничения общая масса случайно менялась бы от уровня к уровню, и полученное торговое время не было бы корректной функцией распределения.

Построение каскада

После определения процедуры выборки множителей построение каскада сводится к простому итеративному процессу. Мы начинаем с одного интервала единичной массы и последовательно разбиваем его на части:

```
//+-----+  
//| GenerateCascade - Build multiplicative cascade measure |  
//+-----+  
bool CSimulationEngine::GenerateCascade(void)  
{  
    double measure[];  
    ArrayResize(measure, 1);  
    measure[0] = 1.0;  
  
    for(int level = 0; level < m_cascade_k; level++)  
    {  
        int n_intervals = ArraySize(measure);  
        int n_children = n_intervals * m_cascade_b;  
  
        double multipliers[];  
        SampleMultipliers(multipliers, n_children);  
  
        double new_measure[];  
        ArrayResize(new_measure, n_children);  
  
        for(int i = 0; i < n_intervals; i++)  
        {  
            double parent_mass = measure[i];  
            for(int j = 0; j < m_cascade_b; j++)  
            {  
                int child_idx = i * m_cascade_b + j;  
                new_measure[child_idx] = parent_mass * multipliers[child_idx];  
            }  
        }  
  
        ArrayResize(measure, n_children);  
        ArrayCopy(measure, new_measure);  
    }  
}
```

```

double total = 0.0;
for(int i = 0; i < ArraySize(measure); i++)
    total += measure[i];

if(total <= 0.0)
{
    Print("SimulationEngine::GenerateCascade - total mass is zero");
    return false;
}

for(int i = 0; i < m_n_points; i++)
    m_measure[i] = measure[i] / total;

return true;
}

```

На уровне 0 имеется один интервал. На уровне 1 — два интервала. На уровне 2 — четыре интервала. После $k = 10$ уровней получаем $2^{10} = 1\,024$ интервала. На каждом уровне масса родительского интервала умножается на случайный вес из распределения, полученного при подгонке. Благодаря последовательному перемножению весов на разных уровнях небольшие исходные асимметрии усиливаются экспоненциально. Интервал с весом выше среднего на уровне 3, вероятно, даст дочерние интервалы с большой абсолютной массой на уровне 4, а у их дочерних интервалов масса будет еще больше. Получается распределение массы с тяжелыми хвостами и кластерами, имитирующее реальную рыночную активность.

После окончательной нормализации сумма значений меры равна ровно 1,0. Получается корректное вероятностное распределение, интегрированием которого можно получить функцию распределения (CDF).

Анимация на рисунке 1 показывает весь процесс от начала до конца. Анимация начинается с одного столбца единичной массы и показывает его разбиение по уровням. Значения множителей приведены явно ($M_L + M_R = 2$) и обеспечивают сохранение массы на каждом шаге. Посмотрите, как первое разбиение вносит небольшую асимметрию, а каждый следующий уровень усиливает ее экспоненциально. К уровню 6 исходный равномерный интервал превращается в гистограмму с острыми пиками: в отдельных интервалах масса на несколько порядков больше, чем в соседних. Накопленная сумма меры дает торговое время $\theta(t)$ — "деформированные часы", которые модуль моделирования использует для изменения временной шкалы FBM.

Рис. 1. Мультипликативный каскад в действии — случайные множители последовательно перемножаются на шести уровнях, превращая равномерную массу в меру с кластерами; ее накопленная сумма задает торговое время

От меры к торговому времени

Торговое время $\theta(t)$ представляет собой функцию распределения каскадной меры:

```
//+-----+
//| IntegrateMeasure - Cumulative sum → trading time theta(t) |
//+-----+
void CSimulationEngine::IntegrateMeasure(void)
{
    m_trading_time[0] = m_measure[0];
    for(int i = 1; i < m_n_points; i++)
        m_trading_time[i] = m_trading_time[i - 1] + m_measure[i];

    double final_val = m_trading_time[m_n_points - 1];
    if(final_val > 0.0)
    {
        for(int i = 0; i < m_n_points; i++)
            m_trading_time[i] /= final_val;
    }
}
```

Получается монотонно возрастающая функция: ее начальное значение близко к нулю, а конечное равно единице. В отличие от прямой диагонали (равномерного хода времени), график $\theta(t)$ имеет крутые участки там, где масса сосредоточена, и плоские там, где массы мало. Когда мы будем выбирать значения FBM в точках, заданных $\theta(t)$, крутые участки обеспечат быстрое продвижение вдоль траектории (большие по модулю доходности), а плоские — считывание почти одинаковых значений FBM (малые по модулю доходности). Этот механизм создает кластеризацию волатильности без явного моделирования условной дисперсии, как в GARCH.

Генерация фракционного броуновского движения

Второй компонент MMAR — фракционное броуновское движение (FBM) с показателем Херста H . В отличие от стандартного броуновского движения ($H = 0,5$) с независимыми приращениями, приращения FBM коррелируют положительно при $H > 0,5$ (персистентность) и отрицательно при $H < 0,5$ (антиперсистентность). Корреляционная структура задается автоковариационной функцией приращений фракционного гауссовского шума (fGn):

$$\text{gamma}(k) = (1/2) * [|k+1|^{(2H)} - 2|k|^{(2H)} + |k-1|^{(2H)}]$$

```
//+-----+
//| FBMAutocovariance - Autocovariance gamma(k) for fGn |
//+-----+
double CSimulationEngine::FBMAutocovariance(int k)
{
    double abs_k = MathAbs(k);
    double two_H = 2.0 * m_H;
    return 0.5 * (MathPow(abs_k + 1.0, two_H)
        - 2.0 * MathPow(abs_k, two_H)
        + MathPow(MathMax(abs_k - 1.0, 0.0), two_H));
}
```

При лаге 0 получаем $\text{gamma}(0) = 1$ (единичная дисперсия каждого приращения). При лаге 1 и $H = 0,55$ получаем $\text{gamma}(1) = 0,5 * (2^{1,1} - 2 + 0) \approx 0,07$ — положительную, но слабую корреляцию. При больших лагах корреляция затухает по степенному закону — поэтому FBM является процессом с долгой памятью. Защитная проверка $\text{MathMax}(\text{abs}_k - 1.0, 0.0)$ обрабатывает случай $k = 0$, когда $|k-1|$ может дать отрицательное основание для возведения в степень.

Все определяется приведенной выше формулой, но *как это выглядит* на практике? На рисунке 2 эта связь показана наглядно. Для трех значений H показаны приращения фракционного гауссовского шума (верхний ряд) и построенная их накопленным суммированием траектория FBM (нижний ряд). При $H = 0,3$ приращения постоянно меняют знак: отрицательные корреляции возвращают процесс назад, формируя изрезанную траекторию с тенденцией к возврату к среднему. При $H = 0,5$ приращения независимы, а траектория представляет собой стандартное случайное блуждание. При $H = 0,7$ положительные корреляции поддерживают движение приращений в одном направлении, формируя плавные трендовые участки. Между тремя столбцами меняются только значения автоковариационной функции $\text{gamma}(k)$, рассчитанные при разных H .

Рис. 2. От приращений к траектории — автоковариация $\gamma(k)$, определяемая N , превращает независимый шум в антиперсистентные, нейтральные или персистентные траектории

Метод Дэвиса — Харта (на основе БПФ)

Алгоритм Дэвиса — Харта генерирует точные выборки fG_n за время $O(n \log n)$ методом циркулянтного вложения. Ковариационную матрицу fG_n можно вложить в более крупную циркулянтную матрицу и диагонализировать ее дискретным преобразованием Фурье (ДПФ). Так генерация коррелированных гауссовских величин сводится к генерации независимых гауссовских величин в частотной области:

```
//+-----+
//| GenerateFBM_DaviesHarte - FFT-based exact FBM synthesis |
//+-----+
bool CSimulationEngine::GenerateFBM_DaviesHarte(void)
{
    int n = m_n_points;
    int two_n = 2 * n;

    //--- Build circulant embedding vector
    double r[];
    ArrayResize(r, two_n);
    for(int k = 0; k < n; k++)
        r[k] = FBMAutocovariance(k);
    for(int k = 1; k < n; k++)
        r[two_n - k] = r[k];

    //--- FFT → eigenvalues of circulant matrix
    complex f[];
    CFastFourierTransform::FFTR1D(r, two_n, f);

    //--- Clamp negative eigenvalues
    int f_size = ArraySize(f);
    for(int i = 0; i < f_size; i++)
    {
        if(f[i].real < 0.0)
            f[i].real = 0.0;
    }
}
```

```

    }

    //--- Generate complex Gaussian noise scaled by sqrt(lambda / 2n)
    complex W[];
    ArrayResize(W, two_n);

    double zr_arr[], zi_arr[];
    MathRandomNormal(0.0, 1.0, two_n, zr_arr);
    MathRandomNormal(0.0, 1.0, two_n, zi_arr);

    for(int i = 0; i < two_n; i++)
    {
        double lam_i = (i < f_size) ? f[i].real : 0.0;
        double scale = MathSqrt(MathMax(lam_i, 0.0) / (double)two_n);
        W[i].real = scale * zr_arr[i];
        W[i].imag = scale * zi_arr[i];
    }

    //--- Inverse FFT
    CFastFourierTransform::FFTC1DInv(W, two_n);

    //--- Extract real parts as fGn increments
    for(int i = 0; i < n; i++)
        m_fbm_increments[i] = W[i].real;

    //--- Cumulative sum → FBM path
    m_fbm[0] = m_fbm_increments[0];
    for(int i = 1; i < n; i++)
        m_fbm[i] = m_fbm[i - 1] + m_fbm_increments[i];

    //--- Center at zero
    double first = m_fbm[0];
    for(int i = 0; i < n; i++)
        m_fbm[i] -= first;

    return true;
}

```

Алгоритм состоит из четырех этапов. Сначала мы строим циркулянтное вложение: зеркально дополняем вектор автоковариации до симметричного вектора длины $2n$. Во-вторых, вычисляем БПФ вектора: действительные части результата — собственные значения циркулянтной матрицы. Мы обнуляем все отрицательные собственные значения (они возникают из-за численных погрешностей и всегда малы). В-третьих, мы генерируем независимые комплексные гауссовские случайные числа и масштабируем каждое на квадратный корень из соответствующего собственного значения, деленного на $2n$. В-четвертых, обратное БПФ возвращает их во временную область, формируя коррелированные приращения fGn . Накопленная сумма преобразует приращения в траекторию, после чего мы центрируем ее относительно нуля.

Мы используем встроенную в MetaTrader 5 реализацию ALGLIB: класс [CFastFourierTransform](#) предоставляет FFTR1D (прямое БПФ из вещественных данных в комплексные) и FFTC1DInv (обратное комплексное БПФ). Функция [MathRandomNormal](#) библиотеки Math\Stat генерирует необходимые гауссовские выборки.

Метод Холецкого (точное разложение)

При небольших n (до 1 024) мы можем явно построить полную ковариационную матрицу и выполнить ее разложение:



```
//+-----+
//| GenerateFBM_Cholesky - Exact FBM via Cholesky decomposition |
//+-----+
bool CSimulationEngine::GenerateFBM_Cholesky(void)
{
    if(m_n_points > 1024)
        return false;

    int n = m_n_points;

    //--- Build covariance matrix
    CMatrixDouble cov;
    cov.Resize(n, n);
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            cov.Set(i, j, FBMAutocovariance(MathAbs(i - j)));

    //--- Regularize diagonal for numerical stability
    for(int i = 0; i < n; i++)
        cov.Set(i, i, cov.Get(i, i) + 1e-10);

    //--- Cholesky factorization
    if(!CTrFac::SPDMatrixCholesky(cov, n, false))
    {
        Print("SimulationEngine::GenerateFBM_Cholesky - decomposition failed");
        return false;
    }

    //--- Multiply L * Z where Z ~ N(0,I)
    double Z[];
    MathRandomNormal(0.0, 1.0, n, Z);

    for(int i = 0; i < n; i++)
    {
        double sum = 0.0;
        for(int j = 0; j <= i; j++)
            sum += cov.Get(i, j) * Z[j];
        m_fbm_increments[i] = sum;
    }

    //--- Cumulative sum → FBM path
    m_fbm[0] = m_fbm_increments[0];
    for(int i = 1; i < n; i++)
        m_fbm[i] = m_fbm[i - 1] + m_fbm_increments[i];

    //--- Center at zero
    double first = m_fbm[0];
    for(int i = 0; i < n; i++)
        m_fbm[i] -= first;

    return true;
}
```

Метод Холецкого математически прост: если C — ковариационная матрица и $C = LL^T$ — ее разложение Холецкого, то LZ (где Z — вектор независимых стандартных нормальных величин) дает вектор с ковариацией, равной в точности C . Диагональная регуляризация $1e-10$ обеспечивает положительную

определенность даже в численно пограничных случаях. Метод [CTrFac::SPDMatrixCholesky](#) из ALGLIB выполняет разложение в той же матрице, сохраняя в ней нижнетреугольный множитель L.

Компромисс между двумя методами очевиден: метод Холецкого имеет сложность $O(n^3)$ по времени и $O(n^2)$ по памяти (построение и разложение полной матрицы), а метод Дэвиса — Харта — $O(n \log n)$ по обоим показателям. При $n = 1024$ метод Холецкого требует порядка миллиарда (10^9) операций и 8 МБ памяти для ковариационной матрицы — достаточно быстро. При $n = 8192$ (для большого горизонта прогноза методом Монте-Карло) метод Холецкого потребовал бы 500 миллиардов операций и 512 МБ памяти. Это явно непрактично, поэтому используется метод Дэвиса — Харта.

Масштабирование по исторической волатильности

Теоретическое стандартное отклонение исходных приращений FBM равно 1,0 (оно следует из автоковариации с единичной дисперсией). Нам нужно изменить их масштаб в соответствии с выборочной волатильностью реальных рыночных данных:

```
//+-----+
//| ScaleFBM - Rescale increments to match target volatility |
//+-----+
void CSimulationEngine::ScaleFBM(void)
{
    double sum = 0.0;
    double sum_sq = 0.0;
    int n = m_n_points;

    for(int i = 0; i < n; i++)
    {
        sum += m_fbm_increments[i];
        sum_sq += m_fbm_increments[i] * m_fbm_increments[i];
    }

    double mean = sum / n;
    double variance = sum_sq / n - mean * mean;
    double current_std = MathSqrt(MathMax(variance, 0.0));

    double scale_factor = (current_std > 0.0) ? m_sample_volatility / current_std : 1.0;

    for(int i = 0; i < n; i++)
    {
        m_fbm_increments[i] *= scale_factor;
        m_fbm[i] *= scale_factor;
    }
}
```

Это простое линейное масштабирование: вычисляем текущее стандартное отклонение сгенерированных приращений, затем умножаем все значения на отношение ($\text{target_vol} / \text{current_std}$). Приращения и траектория масштабируются вместе, чтобы сохранить согласованность между ними. Масштабирование сохраняет корреляционную структуру (умножение коррелированных гауссовских величин на константу меняет дисперсию, но не корреляции) и приводит результат к наблюдаемому уровню волатильности.

Композиция процесса MMAR

На заключительном этапе эти два компонента объединяются. Процесс MMAR задается формулой $X(t) = B_H[\theta(t)]$: мы берем значения траектории FBM в точках, определяемых торговым временем. Именно деформация времени делает модель эффективной: FBM обеспечивает гауссовскую структуру и долгую

память, а торговое время, построенное каскадом, добавляет мультифрактальное масштабирование и кластеризацию волатильности:

```
//+-----+
//| ComposeMMARProcess - Time-deform FBM by trading time |
//+-----+
void CSimulationEngine::ComposeMMARProcess(void)
{
    int n = m_n_points;

    for(int j = 0; j < n; j++)
    {
        double theta_j = m_trading_time[j];
        double fbm_idx = theta_j * (n - 1);

        int idx_lo = (int)MathFloor(fbm_idx);
        int idx_hi = idx_lo + 1;

        if(idx_lo < 0)
            idx_lo = 0;
        if(idx_hi >= n)
            idx_hi = n - 1;
        if(idx_lo >= n)
            idx_lo = n - 1;

        double frac = fbm_idx - (double)idx_lo;

        m_mmar_process[j] = m_fbm[idx_lo] + frac * (m_fbm[idx_hi] - m_fbm[idx_lo]);
    }

    for(int i = 0; i < n - 1; i++)
        m_mmar_returns[i] = m_mmar_process[i + 1] - m_mmar_process[i];
}
```

Для каждого временного шага j мы получаем торговое время $\theta(j)$ — значение от 0 до 1. Преобразуем его в дробный индекс массива FBM: $\text{fbm_idx} = \theta(j) * (n-1)$. Поскольку индекс обычно нецелый, используем линейную интерполяцию между двумя ближайшими значениями FBM. В результате получается гладкий составной процесс без артефактов дискретизации.

Принцип работы интуитивно понятен. На крутых участках графика торгового времени (большая масса в каскаде) соседние значения θ сильно различаются. Поэтому мы делаем большие шаги вдоль траектории FBM и получаем большие по модулю доходности. На плоских участках графика торгового времени (малая масса) соседние значения θ почти одинаковы. Мы вновь получаем почти те же значения FBM, что дает околонулевые доходности. Так каскад создает кластеризацию волатильности: при всплесках активности "рыночные часы" быстро движутся вдоль FBM, а в спокойные периоды едва продвигаются.

На последнем этапе доходности вычисляются как простые первые разности составного процесса. Синтетические доходности обладают тремя ключевыми свойствами рыночных данных: тяжелыми хвостами (мультифрактальный каскад), долгой памятью (FBM при H , отличным от 0,5) и кластеризацией волатильности (деформация времени).

На рисунке 3 этот механизм показан наглядно. Вверху золотой курсор движется вдоль траектории FBM с переменной скоростью: ускоряется на крутых участках графика $\theta(t)$ (большая масса каскада) и замедляется на плоских (малая масса). В среднем ряду составной процесс MMAR строится в реальном времени. Обратите внимание: та же гладкая траектория FBM дает всплески сильных колебаний там, где курсор двигался быстро. В нижнем ряду виден результат: в доходностях отчетливо проявляется

кластеризация волатильности с чередованием спокойных периодов и резких всплесков, хотя явного механизма условной дисперсии, как в GARCH, нет.

Made By MMQ

Рис. 3. Деформация времени в действии — курсор быстро проходит участки FBM, соответствующие большой массе, и медленно проходит участки с малой массой, создавая кластеры волатильности в доходностях составного процесса MMAR

Проверка результатов моделирования

Теперь напомним тестовый скрипт, который проверяет модуль моделирования при заданных параметрах и соответствие результата ожидаемым статистическим свойствам:

```
//+-----+
//|                                     MMAR Simulation Test |
//|                                     Copyright 2026, MMQ - Muhammad Minhas Qamar |
//|                                     https://www.mql5.com/ru/articles/22521 |
//+-----+
#property copyright "Copyright 2026, MMQ - Muhammad Minhas Qamar"
#property link      "https://www.mql5.com/ru/articles/22521"
#property version   "1.00"
#property strict
#property script_show_inputs

#include <MMAR\SimulationEngine.mqh>

input int      InpSeed          = 42;
input double   InpH              = 0.55;
input int      InpCascadeB      = 2;
input int      InpCascadeK      = 10;    // 2^10 = 1024 points
input int      InpDistType      = 0;     // 0=Normal, 1=Binomial, 2=Poisson, 3=Gamma
input double   IniParam0        = 0.6;   // alpha_0 (or alpha_min for binomial)
input double   IniParam1        = 0.0;   // alpha_max (binomial) or gamma (gamma dist)
input double   InpSampleVol     = 0.001;

//+-----+
```

```

//| Script program start function |
//+-----+
void OnStart()
{
    MathSrand(InpSeed);

    PrintFormat("=== MMAR Simulation Engine Test ===");
    PrintFormat("H = %.4f | b = %d | k = %d | N = %d",
        InpH, InpCascadeB, InpCascadeK, (int)MathPow(InpCascadeB, InpCascadeK));
    PrintFormat("Distribution = %d | params = [%.4f, %.4f] | sample_vol = %.6f",
        InpDistType, IniParam0, IniParam1, InpSampleVol);

    double dist_params[4];
    dist_params[0] = IniParam0;
    dist_params[1] = IniParam1;
    dist_params[2] = 0.0;
    dist_params[3] = 0.0;

    CSimulationEngine engine;
    if(!engine.Init(InpH, (ENUM_MMAR_DISTRIBUTION)InpDistType, dist_params,
        InpSampleVol, InpCascadeB, InpCascadeK))
    {
        Print("Init failed");
        return;
    }

    uint start_time = GetTickCount();
    if(!engine.Generate())
    {
        Print("Generate failed");
        return;
    }
    uint elapsed = GetTickCount() - start_time;
    PrintFormat("Generation time: %u ms", elapsed);

    //-- Validate cascade: measure sums to 1, trading time monotonic [0,1]
    double measure[];
    engine.GetMeasure(measure);
    double measure_sum = 0.0;
    double measure_min = DBL_MAX;
    double measure_max = -DBL_MAX;
    for(int i = 0; i < ArraySize(measure); i++)
    {
        measure_sum += measure[i];
        if(measure[i] < measure_min)
            measure_min = measure[i];
        if(measure[i] > measure_max)
            measure_max = measure[i];
    }
    PrintFormat("\n--- Cascade ---");
    PrintFormat("Measure sum: %.10f (should be 1.0)", measure_sum);
    PrintFormat("Measure range: [%.2e, %.2e]", measure_min, measure_max);

    double theta[];
    engine.GetTradingTime(theta);
    bool monotonic = true;
    for(int i = 1; i < ArraySize(theta); i++)

```

```

    {
        if(theta[i] < theta[i - 1])
        {
            monotonic = false;
            break;
        }
    }
    PrintFormat("Trading time: [%.6f, %.6f] | Monotonic: %s",
        theta[0], theta[ArraySize(theta) - 1],
        monotonic ? "YES" : "NO");

//--- Validate FBM
double fbm[];
engine.GetFBM(fbm);
double fbm_min = DBL_MAX;
double fbm_max = -DBL_MAX;
for(int i = 0; i < ArraySize(fbm); i++)
{
    if(fbm[i] < fbm_min)
        fbm_min = fbm[i];
    if(fbm[i] > fbm_max)
        fbm_max = fbm[i];
}
PrintFormat("\n--- FBM ---");
PrintFormat("FBM range: [%.6f, %.6f]", fbm_min, fbm_max);
PrintFormat("FBM[0]: %.10f (should be ~0)", fbm[0]);

//--- Validate MMAR returns
double returns[];
int n_returns;
engine.GetReturns(returns, n_returns);
double vol = engine.GetVolatility();

double ret_sum = 0.0;
double ret_min = DBL_MAX;
double ret_max = -DBL_MAX;
for(int i = 0; i < n_returns; i++)
{
    ret_sum += returns[i];
    if(returns[i] < ret_min)
        ret_min = returns[i];
    if(returns[i] > ret_max)
        ret_max = returns[i];
}
double ret_mean = ret_sum / n_returns;

PrintFormat("\n--- MMAR Returns ---");
PrintFormat("N returns: %d", n_returns);
PrintFormat("Volatility (std): %.6f (target sample_vol: %.6f)", vol, InpSampleVol);
PrintFormat("Mean return: %.8f", ret_mean);
PrintFormat("Return range: [%.6f, %.6f]", ret_min, ret_max);

//--- Quick Hurst check via variance ratio at different scales
PrintFormat("\n--- Variance Ratio (Hurst proxy) ---");
int scales[] = {1, 2, 4, 8, 16, 32};
for(int s = 0; s < ArraySize(scales); s++)
{

```

```

int sc = scales[s];
if(sc >= n_returns)
    break;

int n_agg = n_returns / sc;
double agg_sum = 0.0;
double agg_sum_sq = 0.0;
for(int i = 0; i < n_agg; i++)
{
    double block = 0.0;
    for(int j = 0; j < sc; j++)
        block += returns[i * sc + j];
    agg_sum += block;
    agg_sum_sq += block * block;
}
double agg_mean = agg_sum / n_agg;
double agg_var = agg_sum_sq / n_agg - agg_mean * agg_mean;
PrintFormat("  scale=%2d | var=%.8f | var/scale^(2H)=%.6f (H=%.2f → scale^2H=%.4
            sc, agg_var, agg_var / MathPow(sc, 2.0 * InpH),
            InpH, MathPow(sc, 2.0 * InpH));
}

//--- Test multiple runs to check MC variance
PrintFormat("\n--- Multi-run volatility check (5 runs) ---");
for(int run = 0; run < 5; run++)
{
    CSimulationEngine eng;
    eng.Init(InpH, (ENUM_MMAR_DISTRIBUTION)InpDistType, dist_params,
            InpSampleVol, InpCascadeB, InpCascadeK);
    if(eng.Generate())
        PrintFormat("  Run %d: vol = %.6f", run + 1, eng.GetVolatility());
    else
        PrintFormat("  Run %d: FAILED", run + 1);
}

Print("\n=== Simulation Engine Test Complete ===");
}

```

Тест проверяет три инварианта, обязательных для любой корректно сгенерированной траектории MMAR. Во-первых, сумма значений каскадной меры должна равняться ровно 1,0 (сохранение массы). Во-вторых, торговое время должно монотонно возрастать (функция распределения). В-третьих, траектория FBM должна начинаться с нуля (центрирование). Кроме структурных проверок, мы проверяем близость итоговой волатильности к целевой `sample_volatility`, согласованность отношения дисперсий на разных масштабах агрегации с заданным показателем Херста и согласованность различающихся результатов независимых прогонов (подтверждение случайности и устойчивости).

При запуске получаем следующий вывод:

```

=== MMAR Simulation Engine Test ===
H = 0.5500 | b = 2 | k = 10 | N = 1024
Distribution = 0 | params = [0.6000, 0.0000] | sample_vol = 0.001000
Generation time: 984 ms

--- Cascade ---
Measure sum: 1.0000000000 (should be 1.0)
Measure range: [1.57e-04, 3.79e-03]
Trading time: [0.001208, 1.000000] | Monotonic: YES

```



```

--- FBM ---
FBM range: [-0.012543, 0.009217]
FBM[0]: 0.0000000000 (should be ~0)

--- MMAR Returns ---
N returns: 1023
Volatility (std): 0.000858 (target sample_vol: 0.001000)
Mean return: 0.00004200
Return range: [-0.005461, 0.004721]

--- Variance Ratio (Hurst proxy) ---
scale= 1 | var=0.00000074 | var/scale^(2H)=0.000001
scale= 2 | var=0.00000163 | var/scale^(2H)=0.000001
scale= 4 | var=0.00000349 | var/scale^(2H)=0.000001
scale= 8 | var=0.00000752 | var/scale^(2H)=0.000001
scale=16 | var=0.00001610 | var/scale^(2H)=0.000001
scale=32 | var=0.00003421 | var/scale^(2H)=0.000001

--- Multi-run volatility check (5 runs) ---
Run 1: vol = 0.000881
Run 2: vol = 0.000844
Run 3: vol = 0.000873
Run 4: vol = 0.000900
Run 5: vol = 0.000885

=== Simulation Engine Test Complete ===

```

Все структурные инварианты соблюдены. Сумма значений каскадной меры равна ровно 1,0, а торговое время монотонно возрастает от 0,001 до 1,0. Диапазон меры [1,57e-04; 3,79e-03] показывает, что максимальный вес интервала в 24 раза больше минимального. Именно такая неоднородность создает кластеризацию волатильности. Траектория FBM начинается с нуля и лежит примерно в диапазоне [-0,013; 0,009], что согласуется с масштабированным случайным блужданием.

По отношению дисперсий можно быстро проверить корректность показателя Херста. Для самоподобного процесса с показателем H дисперсия агрегированных доходностей при масштабе s должна расти пропорционально s^{2H} . Если на каждом масштабе разделить измеренную дисперсию на s^{2H} , отношение должно быть примерно постоянным для всех масштабов. В выводе значения действительно почти постоянны (все около 0,000001). Это подтверждает ожидаемую структуру масштабирования сгенерированного FBM при $H = 0,55$.

Итоговая волатильность (0,000858) ниже целевой (0,001000). Это ожидаемо: функция ScaleFBM() приводит волатильность *приращений FBM* к целевой, а последующая деформация времени на основе каскада перераспределяет ее во времени. Одни доходности увеличиваются по модулю (на участках каскада с большой массой), другие уменьшаются (на участках с малой массой), а общее стандартное отклонение немного меняется. В пяти независимых прогонах волатильность меняется от 0,000844 до 0,000900. Это показывает случайную изменчивость результатов Монте-Карло, присущую стохастическому моделированию. Именно поэтому в седьмой части мы проведем сотни или тысячи прогонов, чтобы получить устойчивое распределение прогнозов.

Генерация одной траектории из 1 024 точек занимает 984 мс; основное время приходится на разложение Холецкого (построение и разложение ковариационной матрицы размером 1024x1024). В модуле Монте-Карло седьмой части нужны сотни прогонов и глубина каскада, обеспечивающая достаточно точек для горизонта прогноза. Основным методом станет алгоритм Дэвиса — Харта.

Заключение

В этой статье мы создали CSimulationEngine — генеративное ядро библиотеки MMAR. По параметрам, полученным при анализе разбиений (часть 4) и подгонке спектра (часть 5), модуль строит полные синтетические ценовые траектории в три этапа:

- Мультипликативный каскад. Он создает неравномерное распределение массы на единичном интервале. На каждом уровне интервалы разбиваются на части, которым присваиваются веса из распределения, полученного при подгонке (нормального, биномиального, пуассоновского или гамма-распределения); нормализация обеспечивает сохранение массы. Накопленная сумма задает мультифрактальное торговое время $\theta(t)$.
- Фракционное броуновское движение. Оно задает гауссовский процесс с долгой памятью. Доступны два метода синтеза: метод Дэвиса — Харта (на основе БПФ, $O(n \log n)$) для длинных траекторий и разложение Холецкого (точное, $O(n^3)$) для коротких. Результат масштабируется в соответствии с исторической волатильностью.
- Деформация времени. Она объединяет оба компонента в виде $X(t) = B_H[\theta(t)]$. Линейная интерполяция позволяет выбирать значения FBM в точках, заданных торговым временем, и получать доходности с тяжелыми хвостами, долгой памятью и кластеризацией волатильности.

Модуль моделирования — повторно используемый компонент без внутреннего состояния между запусками: инициализируем его параметрами, генерируем траекторию и получаем результаты. Такая архитектура позволяет легко запускать множество независимых экземпляров параллельно. В седьмой части мы так и поступим: включим модуль в цикл Монте-Карло, выполняющий сотни прогонов для получения распределения прогнозных оценок волатильности с доверительными интервалами.

Программы, представленные в этой статье, предназначены исключительно для образовательных целей. Они не предназначены для реальной торговли и не оптимизированы для работы в реальных рыночных условиях. Код помогает разобраться в методах мультифрактального моделирования. Прежде чем адаптировать его для реальной торговли, обязательно тщательно проверьте работу на демо-счетах.

Как получить исходный код через MQL5 Algo Forge

Все исходные файлы приложены к статье ниже. Полный репозиторий также доступен на [MQL5 Algo Forge](https://www.mql5.com/forge/ayantrader/MMAR) — платформе сообщества на базе Git для обмена торговыми проектами и совместной работы над ними. Получить код можно за два простых шага:

Шаг 1 — создайте форк проекта на Algo Forge. Откройте forge.mql5.io/ayantrader/MMAR в браузере и войдите под своей учетной записью MQL5. Нажмите кнопку **Fork** на странице проекта, введите название и описание своего форка и сохраните изменения. В вашей учетной записи Algo Forge появится личная копия репозитория со всеми файлами, историей коммитов и ветвями.

ayantrader / MMAR

Unwatch

1

Star

1

Fork

0

<> Code

Issues

Pull requests

Projects

Releases

Packages

Wiki

Activity

Actions

...

Repository Part of The "Beyond GARCH" Series On MQL5

forecasting

mql5

python

volatility

Manage topics

6 commits

1 branch

0 tags

111 KiB

main ▾

Find a file

Add file ▾

HTTPS

<https://forge.mql5.io/ayantrader/>

...

Search code...

Exact ▾



ayantrader

1bacfa725d

initial commit

last month



BeyondGARCH

initial commit

last month



Experts/MMAR

initial commit

last month



Include/MMAR

initial commit

last month



Scripts/MMAR

initial commit

last month



LICENSE.md

initial commit

last month



README.md

initial commit

last month

Рис. 4. Создание форка проекта на Algo Forge — зеленая стрелка указывает на кнопку "Fork" на странице проекта

Шаг 2 — клонируйте свой форк в MetaEditor. В MetaEditor откройте **Сервис → Options → Community** и убедитесь, что вошли под той же учетной записью MQL5. Откройте панель "Навигатор", кликните правой кнопкой мыши и нажмите "Обновить". Ваш форк появится в папке Shared Projects. Кликните по проекту правой кнопкой мыши и выберите **Git Clone**, чтобы загрузить его на компьютер. Чтобы проверить, что все скопировано, снова кликните правой кнопкой мыши по папке проекта и выберите **Git Log** — откроется полная история коммитов текущей ветки.

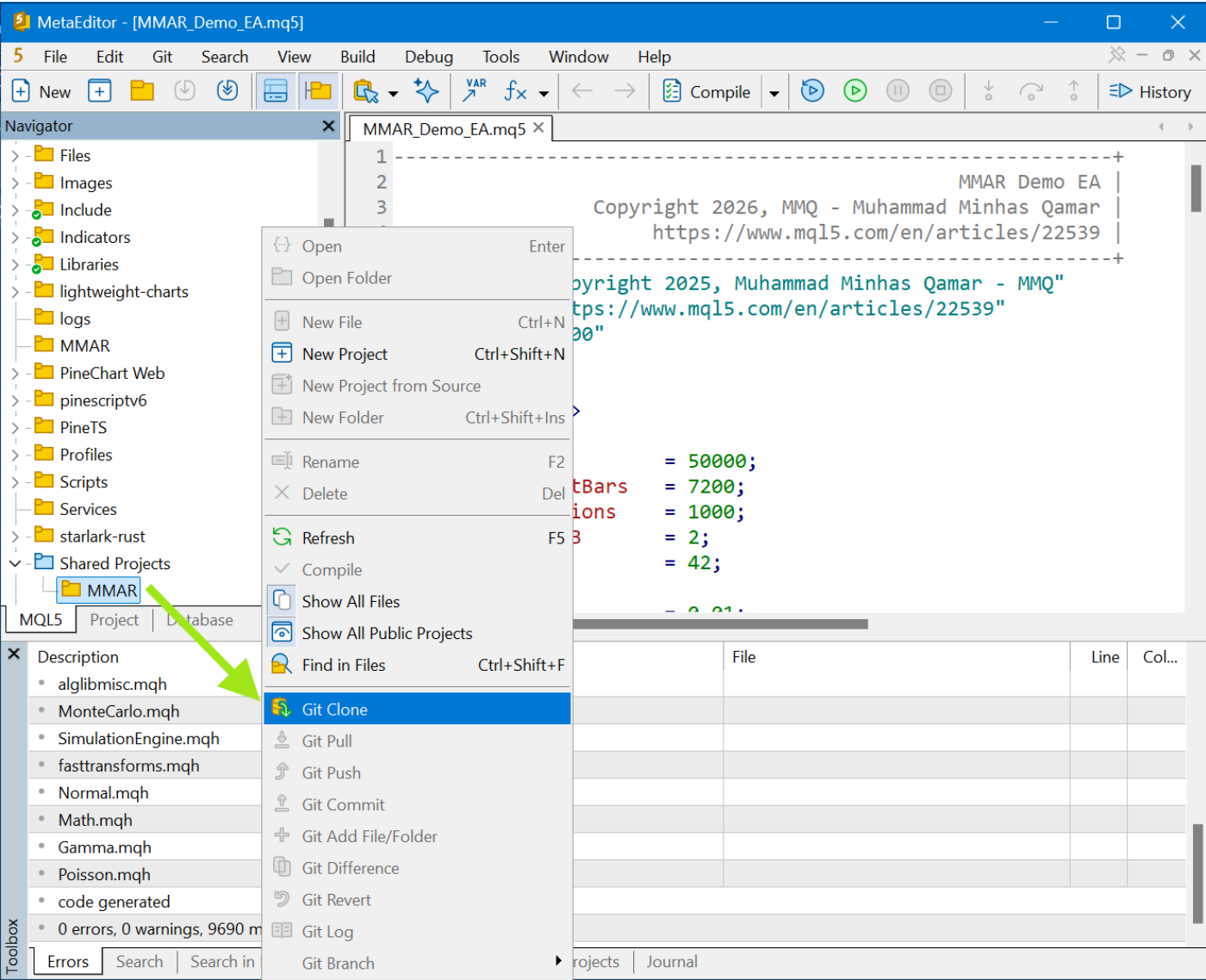


Рис. 5. Клонирование форка в MetaEditor — зеленая стрелка указывает на команду Git Clone в контекстном меню "Навигатор"

Имя файла	Описание
MQL5\Include\MMAR\MMARConstants.mqh	Общие перечисления, структуры и определения типов библиотеки MMAR (часть 4)
MQL5\Include\MMAR\OLS.mqh	Регрессия методом наименьших квадратов через псевдообратную матрицу на основе SVD с полным набором диагностических показателей (часть 4)
MQL5\Include\MMAR\HurstGHE.mqh	Оценщик обобщенного показателя Херста, резервный метод (часть 4)
MQL5\Include\MMAR\PartitionAnalysis.mqh	Мультифрактальный анализ функции разбиения с диагностикой масштабирования (часть 4)
MQL5\Include\MMAR\SpectrumFitter.mqh	Вычисление мультифрактального спектра и подгонка к четырем распределениям методом оптимизации BLEIC (часть 5)
MQL5\Include\MMAR\SimulationEngine.mqh	Модуль моделирования MMAR: мультипликативный каскад, генерация FBM (метод Дэвиса — Харта и разложение Холецкого) и композиция с деформацией времени (часть 6)
MQL5\Scripts\MMAR\MMAR_PartitionTest.mq5	Тестовый скрипт для анализа разбиений на реальных рыночных данных (часть 4)

Имя файла	Описание
MQL5\Scripts\MMAR\MMAR_SpectrumTest.mq5	Тестовый скрипт полного пайплайна анализа разбиений и подгонки спектра на реальных рыночных данных (часть 5)
MQL5\Scripts\MMAR\MMAR_SimulationTest.mq5	Тестовый скрипт для проверки инвариантов каскада, свойств FBM и согласованности результатов нескольких прогонов (часть 6)

Следующая статья >>

Muhammad Minhas Qamar

[Пакистан](#) [9657](#)

Developer by Profession, Trader by Hobby

Gmail: ayanminhasshayar@gmail.com

Другие статьи автора

- [За пределами GARCH \(Часть VII\): Прогнозирование волатильности методом Монте-Карло в MQL5](#)
- [За пределами GARCH \(Часть V\): Подгонка мультифрактального спектра в MQL5](#)
- [За пределами GARCH \(Часть IV\): Реализация анализа разбиений в MQL5](#)
- [За пределами GARCH \(Часть III\): Построение MMAR и вердикт](#)
- [За пределами GARCH \(Часть II\): Оценка фрактальной размерности рынков](#)
- [За пределами GARCH \(Часть I\): Сравнение моделей MMAR Мандельброта и GARCH Энгла](#)

[Подпишитесь](#) и узнавайте о
популярных статьях и кодах